Chương 12: Mô hình tùy chỉnh và Huấn luyện với TensorFlow

Giảng viên: TS. Trần Thành Thắng
Đại học Đông Á
Năm học 2024-2025

Gemini Notebook

Nội dung chương

1. Giới thiệu TensorFlow & Kiến trúc
2. Sử dụng TensorFlow như NumPy
3. Tùy chỉnh mô hình và Thành phần (Custom Components)
4. Hàm và biểu đồ TensorFlow (AutoGraph)
5. Vòng lặp Huấn luyện Tùy chỉnh

Gemini Notebook

Giới thiệu nhanh về TensorFlow

Thư viện mã nguồn mở cho học máy quy mô lớn và học sâu. Cốt lõi tương tự NumPy, nhưng thao tác với **Tensor** thay vì mảng.

Tính toán phân tán hàng ngàn thiết bị. Hỗ trợ phần cứng tăng tốc mạnh mẽ: **GPU** (Graphics Processing Unit) và **TPU** (Tensor Processing Unit).

Chu vện Tự dịch biểu thức tính toán Python tiành **TPU** và Tensor Processing Unit.

Tự động biên dịch biểu thức tính toán Python thành **Biểu đồ (Graph)** C++ hiệu suất cao.

Gemini Notebook

Hệ sinh thái TensorFlow

TensorBoard

Trực quan hóa dữ liệu, đồ thị mô hình và phân tích quá trình huấn luyện.

TensorFlow Hub

Nơi tải xuống và sử dụng lại các mô hình đã huấn luyện (Pretrained models) giúp đẩy nhanh tiến độ.

TensorFlow Extended (TFX)

Bộ công cụ xây dựng quy trình ML toàn diện và triển khai mô hình lên môi trường sản xuất (Production).

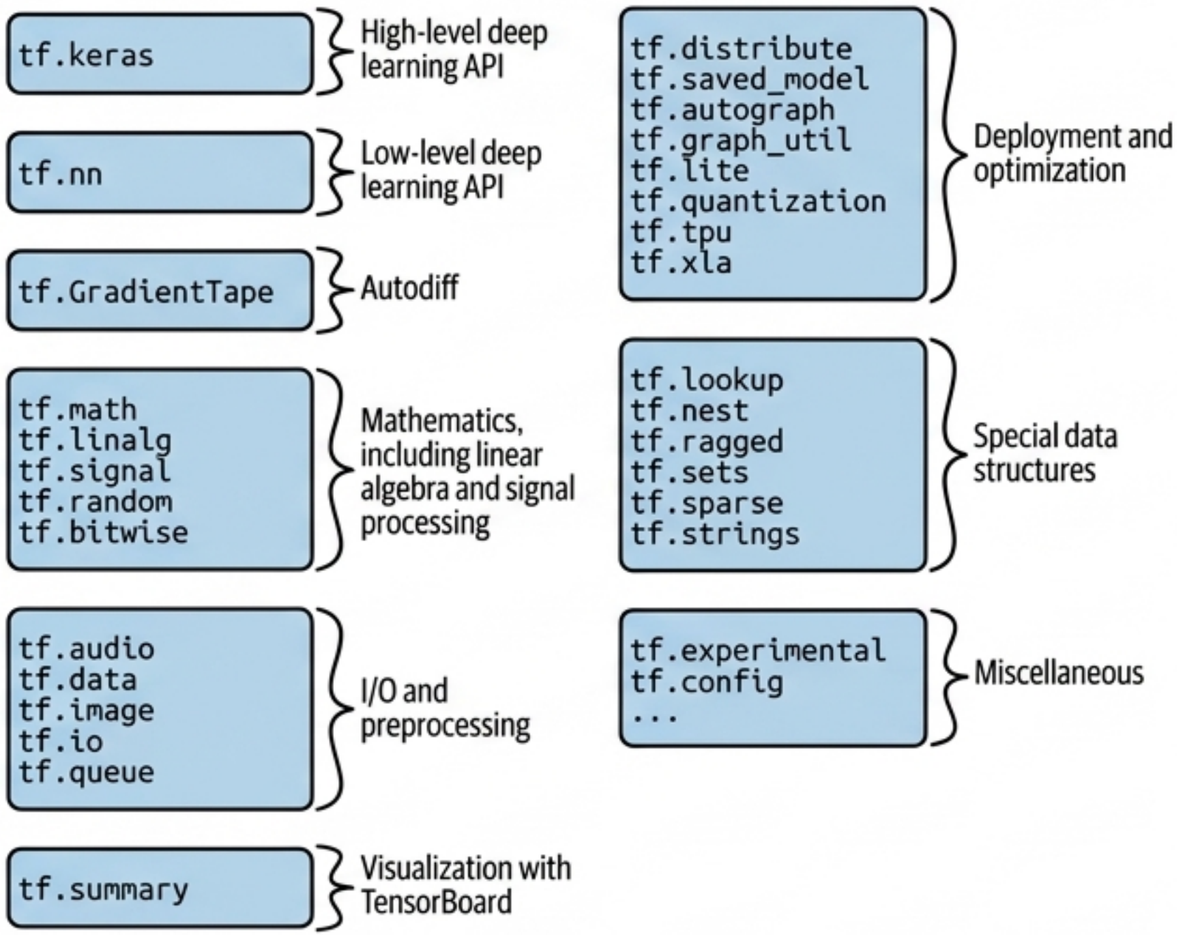
TF Lite & TF.js

Triển khai mô hình trên di động (Android, iOS), phần cứng nhúng (Raspberry Pi) và ngay trên trình duyệt Web bằng JavaScript.

Gemini Notebook

Tổng quan về API Python của TensorFlow (1/2)

- API Python thiết kế phân tầng từ thấp đến cao.
- **Cấp thấp (Nền tảng):** Phép toán trên Tensor, Biến số (Variables), Tự động vi phân (**tf.GradientTape**).
- **Cấp cao:** Keras (**tf.keras**) cung cấp API trực quan xây dựng lớp (layers) và mô hình (models).
- **Dữ liệu:** API dữ liệu (**tf.data**) nạp, xử lý trước và cung cấp dữ liệu huấn luyện hiệu quả.



Hình 12-1: Tổng quan về API Python của

Gemini Notebook

Tổng quan về API Python của TensorFlow (2/2)

Người học mới (Keras)

Sự trừu tượng hóa mức độ phức tạp giúp tiếp cận nhanh chóng thông qua **Keras**.

Nhà nghiên cứu (Low-level API)

Viết các thuật toán huấn luyện phi tiêu chuẩn (như trong **GANs** hay **Meta-learning**) bằng các **API cấp thấp**.

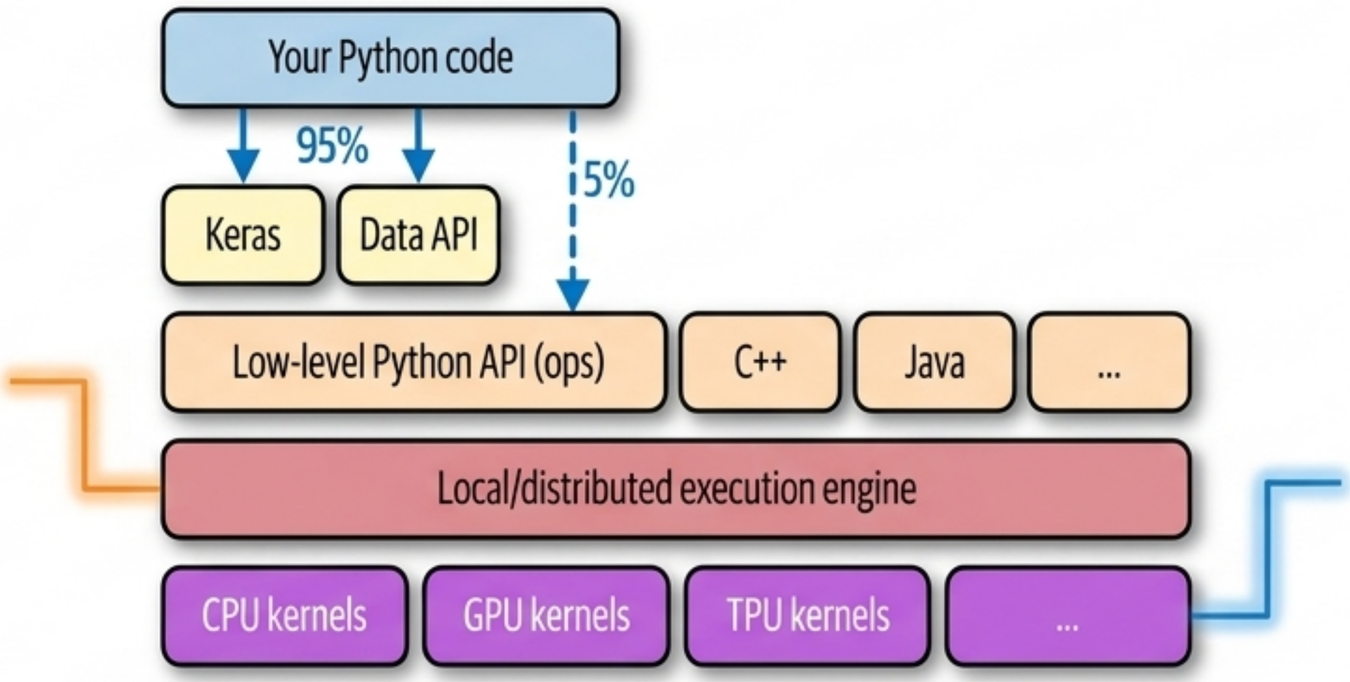
Tổng kết: Hình 12-1 tóm tắt bao quát các module quan trọng. Chúng ta sẽ khám phá từ **Tensor cơ bản** đến **tùy chỉnh lớp/mô hình** qua API này.

Gemini Notebook

Kiến trúc của TensorFlow (1/2)

Hình 12-2 minh họa kiến trúc bên trong của TensorFlow.

Execution Engine (Trung tâm): Lớp trung tâm thực hiện các biểu đồ (Graphs) bằng C++ (mã nguồn lõi tối ưu cực cao).



Đa nền tảng (Bên dưới): Khả năng chạy đồng thời (multi-threading) trên nhiều loại phần cứng (CPU, GPU, TPU) và qua mạng (Android, iOS, Cloud).

Hình 12-2: Kiến trúc của TensorFlow

Gemini Notebook

Kiến trúc của TensorFlow (2/2)

Đa ngôn ngữ

API giao diện: Python (phổ biến nhất), C++, Java, Go, Swift. Tương tác chủ yếu qua **Keras** (High-level API).

Lớp XLA (Accelerated Linear Algebra)

Trình biên dịch mạnh mẽ, tối ưu hóa **Graph**. Kết hợp toán tử (kernel fusion) để tăng tốc tối đa phần cứng.

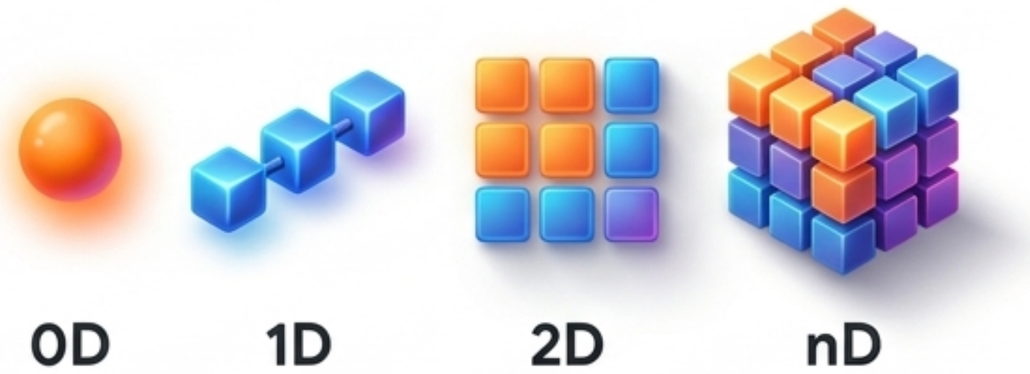
TF Distributed Execution

Phân phối trực tiếp tính toán trên nhiều máy vật lý (**Parameter Server**, **MirroredStrategy**).

Gemini Notebook

Tensor và Phép toán (Tensors and Operations)

- **Định nghĩa:** **Tensor** là cấp số liệu đa chiều kế thừa từ Toán học: vô hướng (scalar - 0D), vectơ (1D), ma trận (2D), không gian đa chiều (nD).
- **Khởi tạo:** Cách dễ nhất là dùng **tf.constant()**.

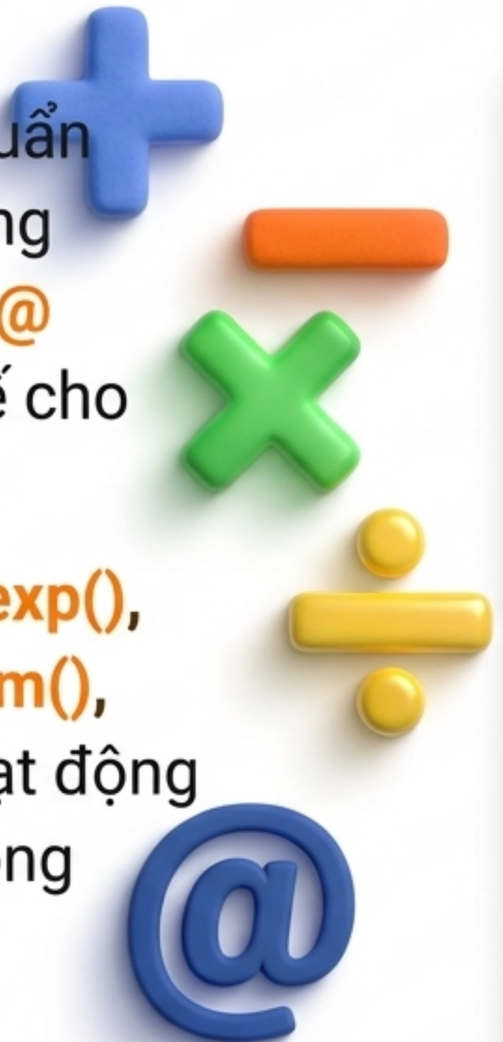



```
import tensorflow as tf

# Tao tensor tu ma tran 2 chieu (2D)
t = tf.constant([[1., 2., 3.], [4., 5., 6.]])
print(t.shape) # In ra: TensorShape([2, 3])
print(t.dtype) # In ra: <dtype: 'float32'>
```

Gemini Notebook

Các hàm toán học cơ bản

- Tính năng: Toán tử chuẩn **Python** được nạp chồng (**overloaded**). Ký hiệu **@** **(Python 3.5+)** thay thế cho **tf.matmul**.
- Các hàm phổ biến: **tf.exp()**, **tf.sqrt()**, **tf.reduce_sum()**, **tf.reduce_mean()**. Hoạt động trực tiếp không cần vòng lặp for.



```
# Tinh toan co ban bang TF operations
t1 = t + 10 # Phép cong (hoạt dọng broadcasting)
t2 = tf.square(t) # Bình phương tưng phan tư (element-wise)
t3 = t @ tf.transpose(t) # Nhan ma tran (Matrix multiplication)
```

Gemini Notebook

Tensors và NumPy: Tương tác qua lại

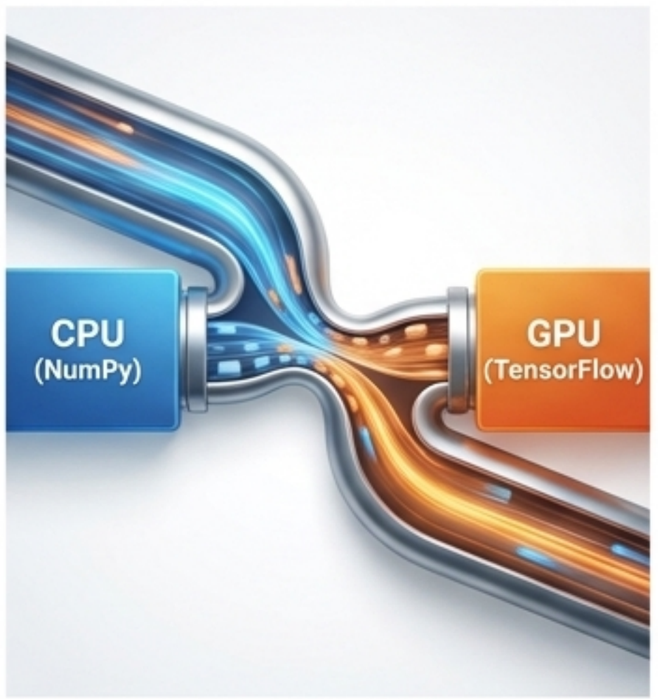
Đặc tính: **Tương tác thân thiện, liền mạch. Chuyển đổi trực tiếp** và **tính toán kết hợp (Cross-computation)** dễ dàng.

Terminal Box

```python
import numpy as np


# Chuyen Numpy -> Tensor
a = np.array([2., 4., 5.])
t_a = tf.constant(a)

# Chuyen Tensor -> Numpy
a_back = t_a.numpy() # Hoac dung np.array(t_a)

# Tinh toan ket hợp (Cross-computation)
t_b = tf.square(a) # TF chap nhan Numpy array
a_b = np.square(t_a) # Numpy chap nhan TF Tensor
```

Gemini Notebook

Lưu ý khi dùng chung Numpy và TensorFlow



- **Nghẽn cổ chai (Bottleneck):** **NumPy** chỉ chạy trên **CPU**. Nếu gọi hàm NumPy cho chạy trên CPU. Nếu gọi hàm NumPy cho **Tensor** đang ở **GPU**, TensorFlow phải sao chép dữ liệu từ **GPU** về CPU → **Chi phí truyền tải lớn.**
- **Giải pháp:** Ưu tiên dùng hàm **TensorFlow** để tận dụng tốc độ **GPU**.
- **Khác biệt kiểu dữ liệu mặc định:**
  - **NumPy:** Dùng **float64** (độ chính xác cao).
  - **TensorFlow:** Dùng **float32** hoặc **bfloat16** (mạng Neural ưu tiên hiệu suất).

Gemini Notebook

Chuyển đổi kiểu dữ liệu (Type Conversions)

Nguyên tắc: **TensorFlow không tự động ép kiểu** như Python/NumPy. Ép kiểu giữa **float32/float64** hoặc **float32/int32** có **chi phí cao**.

Lỗi TypeError

Problem & Solution

tf.cast

```
# Kieu int32 khong the cong voi float32 ngam dinh:
# tf.constant(2.0) + tf.constant(40) # --> Loi TypeError

# Can phai ep kieu chu dong va ro rang bang tf.cast:
t2 = tf.constant(40., dtype=tf.float64)

# tf.cast(t2, tf.float32) ha do chinh xac xuong float32
result = tf.constant(2.0) + tf.cast(t2, tf.float32)
```

Gemini Notebook

Biến số (Variables)

Vấn đề: `tf.constant()` có tính **immutable** (không thể thay đổi). Nhưng **mạng nơ-ron** cần liên tục cập nhật trọng số (weights) và độ lệch (biases) qua mỗi chu kỳ học (epoch).

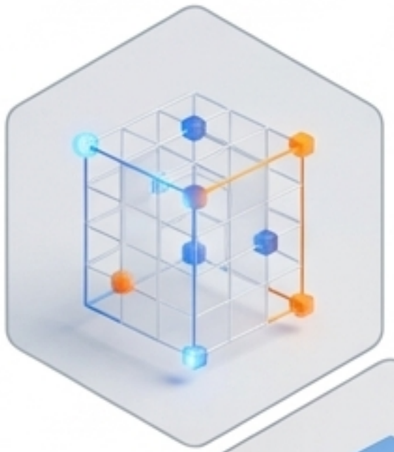
Giải pháp: Sử dụng `tf.Variable`.

Terminal Box

```
v = tf.Variable([1.,2., 3.], [4., 5., 6.]])

# Cap nhat toan bo bien (dung phuong thuc assign)
v.assign(2 * v)

# Cap nhat vao dung 1 vi tri cu the (nhu numpy arrays)
v[0, 1].assign(42)

# Cap nhat toan cục bang phep cong (+)
v.assign_add([1., 1., 1.], [1., 1., 1.]])
```

Gemini Notebook

Các cấu trúc dữ liệu nâng cao khác



Sparse tensors (tf.SparseTensor): Tối ưu **bộ nhớ** cho ma trận chứa phần lớn **số 0**. Lưu 3 mảng: **indices**, **values**, **dense_shape**.

Tensor arrays (tf.TensorArray): Danh sách Tensors cho phép **append động**. Hữu ích cho mạng tuần tự **RNN**.

Ragged tensors (tf.RaggedTensor): Mảng **bất đối xứng** (VD: các chuỗi con độ dài không bằng nhau).

String tensors: Lưu trữ kiểu chữ, chuỗi **UTF-8** hoặc **byte arrays**, cần thiết cho **NLP**.

Sets: Tập hợp Toán học dưới dạng **Sparse tensor** qua tf.sets.

Gemini Notebook

Keras API cung cấp sẵn hàng chục hàm losses, metrics, layers, optimizers chuẩn (ví dụ: `loss="mse"`, `activation="relu"`). Thực tế phức tạp hơn:

1. Bài toán đặc thù yêu cầu hàm tính sai số hoàn toàn mới (VD: Phạt lỗi sai số dương nặng gấp đôi sai số âm).

2. Nghiên cứu tạo ra một **cấu trúc** Tầng Nơ-ron (Layer) chưa từng có.

3. Xây dựng **mạng đa luồng:** 3 nguồn dữ liệu vào (Multi-input) và 2 luồng kết quả xuất ra (Multi-output).

TensorFlow 2 cho phép dùng Python viết mọi thứ, đóng gói lại và dùng như Module nguyên bản.

Gemini Notebook

Tùy chỉnh Hàm mất mát (Custom Loss) - Huber Loss (1/2)

Ví dụ: Bài toán dự đoán giá cả. Lỗi (error = label - prediction).

MSE (Mean Squared Error): Phạt bình phương lỗi -> rất nhạy cảm nhiễu.
MAE (Mean Absolute Error): Trừng phạt tuyến tính -> không mượt quanh điểm 0.
Huber Loss: Hành vi giống MSE khi lỗi nhỏ, giống MAE khi lỗi lớn.

```python
def huber_fn(y_true, y_pred):
    error = y_true - y_pred
    is_small_error = tf.abs(error) < 1
    squared_loss = tf.square(error) / 2
    linear_loss = tf.abs(error) - 0.5
    # Dung tf.where() giong nhu np.where()
    return tf.where(is_small_error , squared_loss , linear_loss)
```

Gemini Notebook

Tùy chỉnh Hàm mất mát (Custom Loss) - Sử dụng (2/2)

```
# Truyen truc tiep ten ham huber vao phuong thuc compile
model.compile(loss=huber_fn , optimizer="nadam", metrics=["mae"])

# Huan luyen nhu binh thuong
model.fit(X_train, y_train, epochs=2)
```

Keras sẽ truyền y_true và y_pred (Tensor) vào huber_fn tự động trong mỗi batch. Tính sai số -> tự động truy vết lại để tính đạo hàm cập nhật ngược (backpropagation).

Gemini Notebook

Lưu ý khi tải (load) mô hình chứa Custom Loss

Vấn đề

Cấu trúc Keras không lưu trữ mã nguồn Python của hàm huber_fn. Khi dùng `keras.models.load_model()` ở script khác, Keras sẽ báo lỗi **"Unknown loss function"**.

Giải pháp

```
# Luu nhu binh thuong
model.save("my_model_with_a_custom_loss.h5")

# Khi load, ban BAT BUOC phai truyen tu dien (dictionary)
# khai bao mapping tu chuoi sang doi tuong Python
model = keras.models.load_model(
    "my_model_with_a_custom_loss.h5",
    custom_objects={"huber_fn": huber_fn}
)
```

Gemini Notebook

Xây dựng Custom Loss bằng OOP (Subclassing) (1/2)

Nếu hàm Loss yêu cầu siêu tham số (như **threshold=1.0** không cố định), sử dụng Hàm Python rất bất tiện. Giải pháp OOP: Kế thừa lớp **keras.losses.Loss**. Khai báo **get_config()** để tự động lưu tham số.

```
class HuberLoss(keras.losses.Loss):
    def __init__(self, threshold=1.0, **kwargs):
        self.threshold = threshold
        super().__init__(**kwargs)

    def call(self, y_true, y_pred):
        error = y_true - y_pred
        is_small_error = tf.abs(error) < self.threshold
        squared_loss = tf.square(error) / 2
        linear_loss = self.threshold * tf.abs(error) - self.threshold**2 / 2
        return tf.where(is_small_error , squared_loss , linear_loss)
```

Gemini Notebook

Xây dựng Custom Loss bằng OOP (Subclassing) (2/2)

Cung cấp `get_config()` để lưu trữ tham số mạnh mẽ hơn.

```
    def get_config(self):
        base_config = super().get_config()
        # Cap nhat thong so cua minh kem theo thong so cha
        return {**base_config , "threshold": self.threshold}

# Chi can tao doi tuong thay vi day string
model.compile(loss=HuberLoss(2.), optimizer="nadam")

# Keras goi get_config de luu threshold=2. vao JSON Model.
model.save("my_custom_model_v2.h5")

# Khi load, custom_objects van la bat buoc!
model = keras.models.load_model("my_custom_model_v2.h5",
    custom_objects={"HuberLoss": HuberLoss})
```

Gemini Notebook

Tùy chỉnh Hàm Đánh giá (Custom Metrics)

Bạn có thể tái sử dụng Custom Loss cho Metric. Khác biệt: Loss dùng để huấn luyện, Metric in ra để đánh giá.

```
model.compile(loss="mse", optimizer="nadam", metrics=[huber_fn])
```

Vấn đề:
Metrics tiêu chuẩn Keras là 'Streaming metrics', tính luỹ kế qua các batch rồi chia trung bình. Hàm Python đơn giản sẽ bị tính sai luỹ kế.

Giải pháp nâng cao:
Kế thừa `keras.metrics.Metric`, quản lý các biến trạng thái bằng `self.add_weight()` và ghi đè hàm `update_state()`, `result()`.

Gemini Notebook

Tùy chỉnh Lớp ẩn không trạng thái (Stateless Custom Layers)

Khái niệm cốt lõi (Core Concept)

Sử dụng khi cấu trúc phi tuyến đặc biệt không chứa tham số cấu hình cần học (Weights/Biases). Keras cung cấp keras.layers.Lambda.

Ứng dụng: Đóng vai trò làm activation function hoặc preprocessing.

```
# Tao tang mu (exponential layer)
exponential_layer = keras.layers.Lambda(lambda x: tf.exp(x))

# Tích hợp vao Sequential Model
model = keras.models.Sequential([
    keras.layers.Dense(30, activation="relu"),
    keras.layers.Dense(1),
    exponential_layer
])
```

Gemini Notebook

Tùy chỉnh Lớp ẩn có trạng thái (Stateful Custom Layer) (1/2)

Khái niệm cốt lõi (Core Concept)

Để tạo Layer riêng có khả năng 'Học' hệ số W, b, kế thừa keras.layers.Layer. Gồm 3 hàm cốt lõi: __init__, build() (khởi tạo Weights khi nhận dạng Shape) và call() (thực thi Feed-Forward).

```
class MyDense(keras.layers.Layer):
    def __init__(self, units, activation=None, **kwargs):
        super().__init__(**kwargs)
        self.units = units
        self.activation = keras.activations.get(activation)

    def build(self, batch_input_shape):
        self.kernel = self.add_weight(
            name="kernel", shape=[batch_input_shape[-1], self.units],
            initializer="glorot_normal")
```

Gemini Notebook

Tùy chỉnh Lớp ẩn có trạng thái (Stateful Custom Layer) (2/2)

```
    # Tiep tuc ham build
    self.bias = self.add_weight(
        name="bias", shape=[self.units], initializer="zeros")
    super().build(batch_input_shape)

    def call(self, X):
        return self.activation(X @ self.kernel + self.bias)

    def compute_output_shape(self, batch_input_shape):
        return tf.TensorShape(batch_input_shape.as_list()[:-1] +
          [self.units])
```

Takeaway note:
Bằng cách này, Layer tùy biến có thể nối chuỗi trong Sequential một cách trong suốt.

Gemini Notebook

Tùy chỉnh toàn bộ Mô hình (Custom Models)

Khi xây dựng kiến trúc không dạng tuần tự (VD: mạng lưới có đường chéo, Residual Neural Networks), cấu trúc Sequential trở nên cồng kềnh.

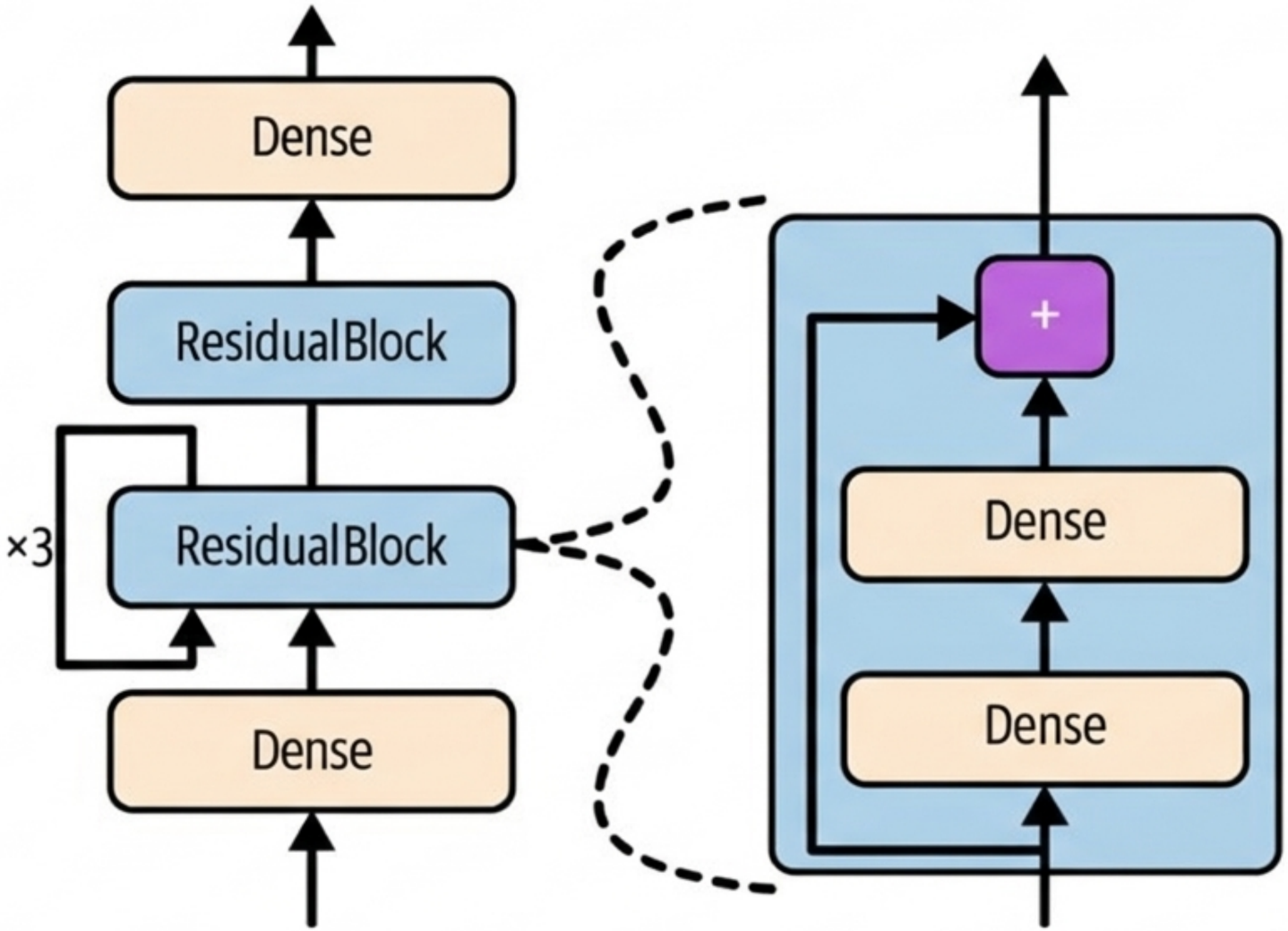
Phương pháp Model Subclassing: Cho phép ta thiết kế bằng Python hoàn toàn tự do.

Điểm mạnh: Có thể dùng luồng điều hướng if...else, vòng lặp for để nối các module một cách linh động (dynamic).

Hướng tiếp cận: Ta sẽ trình bày mô hình mẫu sử dụng một lớp khối (Residual Block) với khả năng Skip Connection.

Gemini Notebook

Phác thảo cấu trúc một kiến trúc tùy ý với kỹ thuật Skip connection. Đường đi thẳng truyền trực tiếp Input cộng với Output của chuỗi tầng ẩn. Mục đích: Giải quyết triệt để lỗi mất gradient (Vanishing Gradient) trong Deep Learning.



Hình 12-3: *Ví dụ mô hình tùy chỉnh: một mô hình tùy ý với...*

Gemini Notebook

Mã Python Xây dựng Residual Block

Viết một Custom Layer đóng vai trò chứa chuỗi Dense và làm phép tính + kết nối.

```
class ResidualBlock(keras.layers.Layer):
    def __init__(self, n_layers , n_neurons , **kwargs):
        super().__init__(**kwargs)
        self.hidden = [keras.layers.Dense(n_neurons , activation="elu")
                       for _ in range(n_layers)]

    def call(self, inputs):
        Z = inputs
        # Vong lap for feed-forward
        for layer in self.hidden:
            Z = layer(Z)
        # Phép Cong Skip Connection
        return inputs + Z
```

Gemini Notebook

Xây dựng Custom Model sử dụng Residual Block

Kế thừa keras.Model giống kế thừa Layer, cấu trúc __init__ và call() tương tự.

```
class ResidualRegressor(keras.Model):
    def __init__(self, output_dim , **kwargs):
        super().__init__(**kwargs)
        self.hidden1 = keras.layers.Dense(30, activation="elu")
        self.block1 = ResidualBlock(2, 30)
        self.block2 = ResidualBlock(2, 30)
        self.out = keras.layers.Dense(output_dim)

    def call(self, inputs):
        Z = self.hidden1(inputs)
        for _ in range(1 + 3): # 4 lan di qua Block 1
            Z = self.block1(Z)
        Z = self.block2(Z)     # 1 lan di qua Block 2
        return self.out(Z)
```

Gemini Notebook

Model Subclassing vs Functional API

Subclassing API

- ⚙️ Cung cấp sự linh hoạt cực lớn (có thể dùng vòng for, luồng logic động).
- ⚠️ Nhược điểm chí mạng: Keras không thể lưu đồ thị model.summary() hoặc vẽ plot_model() khi chưa truyền thử bộ dữ liệu (Model is not Built). Lỗi logic luồng xử lý bên trong Python rất khó phát hiện.

Lời khuyên (Takeaway): Hãy dùng Functional API nếu có thể, trừ phi mạng có vòng lặp động hoặc điều kiện tùy thuộc dữ liệu sinh ra (dynamic branching).

Gemini Notebook

Chế độ Biểu đồ (Graph - TF 1.x)

- Yêu cầu định nghĩa đồ thị trước.
- Mở `tf.Session` để thực thi (Graph Execution).
- Quá trình debug vô cùng tàn bạo.

Chế độ Tức thời (Eager - TF 2.x)

- Phương thức hoạt động mặc định hiện tại.
- Thực thi tức thì: Mã chạy dòng nào có kết quả ngay.
- Gỡ lỗi dễ dàng bằng `print()` truyền thống.

Khuyết điểm của Eager Mode: Chậm hơn nhiều so với Graph do không thể tối ưu song song hóa phần cứng.

Gemini Notebook

`@tf.function”: Phép màu của AutoGraph

TensorFlow 2 mang đến giải pháp dung hòa bằng AutoGraph! Bạn viết code Python bình thường, chỉ cần đánh dấu `@tf.function`. TF sẽ tự động dịch thành đồ thị C++ ẩn bên dưới.

```python
def cube(x):
    return x ** 3

# Tinh toan - Nhanh cho Test nhưng cham khi huan luyện
cube(tf.constant(2.0))

# Khai bao TF Function bang decorator
@tf.function
def tf_cube(x):
    return x ** 3

# tf_cube tro thanh mot computational graph!
tf_cube(tf.constant(2.0))
```

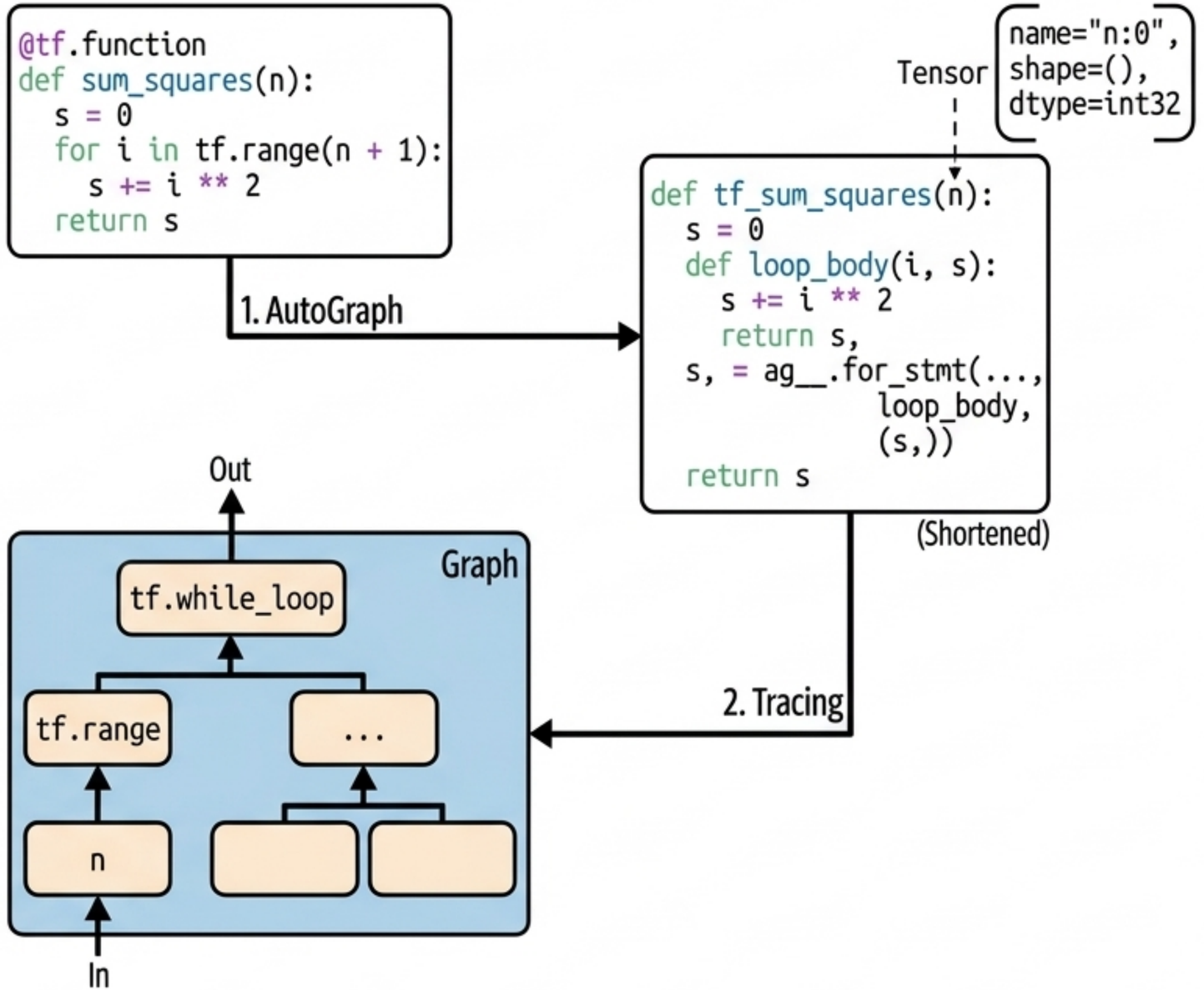
Cầu nối chuyển đổi tự động sang Graph C++

Gemini Notebook

Cách AutoGraph tạo Biểu đồ và Tracing (1/2)

Mã Python được quét, tạo Abstract Syntax Tree (Cây cú pháp trừu tượng).

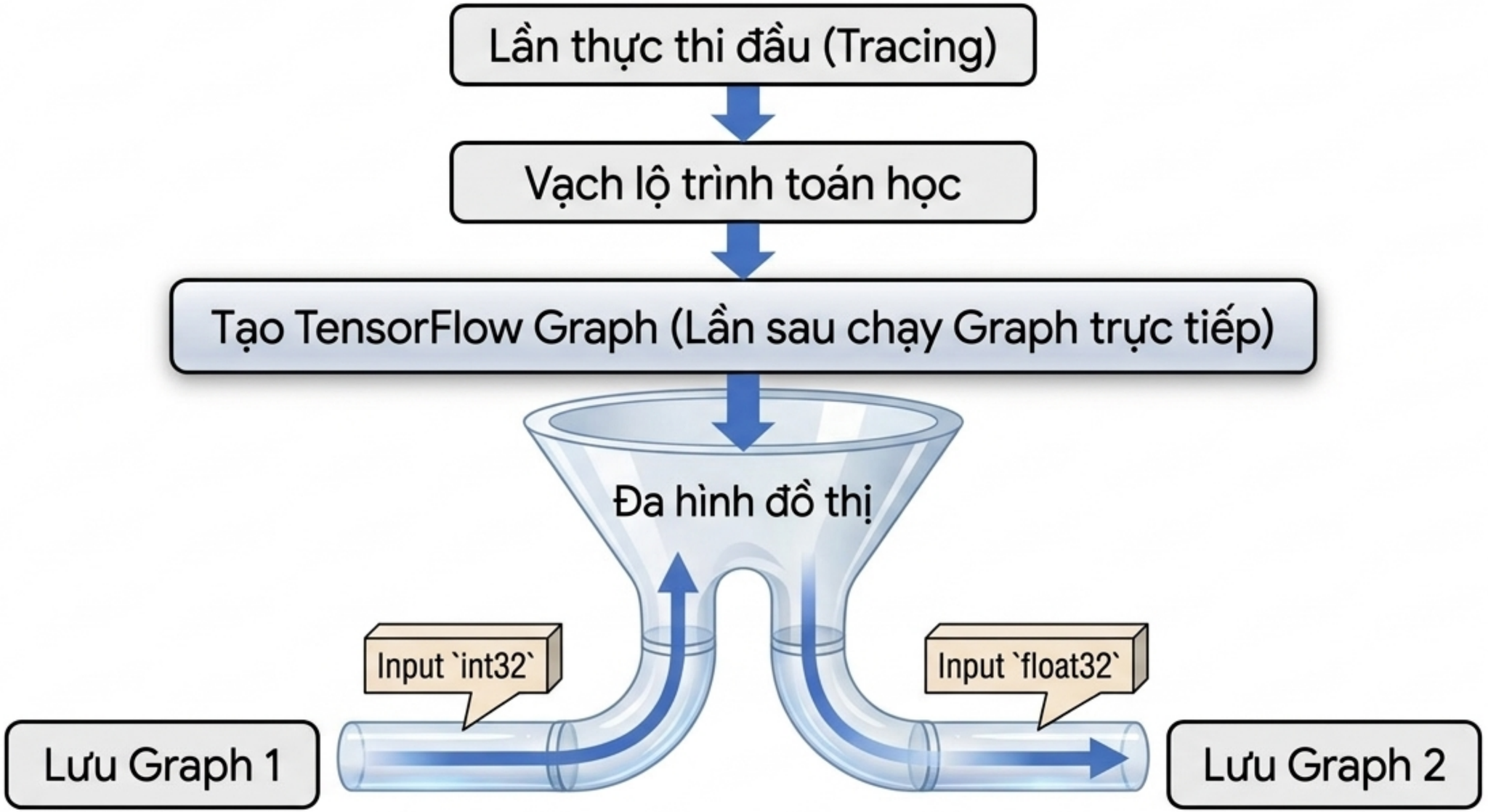
Sinh ra hàm Python nâng cấp gọi các toán tử điều khiển lõi của TF như `**tf.cond**, `**tf.while_loop**`.



*Hình 12-4: Cách TensorFlow tạo biểu đồ bằng AutoGraph và

Gemini Notebook

Cách AutoGraph tạo Biểu đồ và Tracing (2/2)



Gemini Notebook

Giới hạn và Quy tắc sống còn của TF Functions (1/2)

Side effects: Hàm `print()` hoặc thao tác IO chỉ chạy một lần duy nhất ở giai đoạn **Tracing**, không nằm trong Đồ thị.

```python
@tf.function
def my_func(x):
    print("Ham dang duoc trace:", x.dtype)
    return x + 1

my_func(tf.constant(1.0))  # In log, Tao Graph float32
my_func(tf.constant(2.0))  # KHONG in log, Tai sụ dung Graph float32
my_func(tf.constant(10))   # In log, Tao Graph int32
```

Biểu đồ đã được trace, hàm Python **KHÔNG được chạy lại!**

Muốn in log động? Khuyên dùng `tf.print()`.

Gemini Notebook

Code Lab

Giới hạn và Quy tắc sống còn của TF Functions (2/2)

Biến ngoài hàm / List Python:

`list.append()` bị "đóng băng" ở trace.

Giải pháp: Hãy dùng `tf.TensorArray`.

```python
@tf.function
def add_random_noise(x):
    # SAI LAM: Gia tri ngau nhien bi dong bang
    # noise = np.random.randn()

    # CHUAN XAC: Gia tri ngau nhien sinh ra moi khi goi Graph
    noise = tf.random.normal(shape=[])
    return x + noise
```

Giá trị ngẫu nhiên bị đóng băng

Giá trị sinh ra mới mỗi khi gọi Graph

Gemini Notebook

Code Lab

Vòng lặp huấn luyện tùy chỉnh

Sự Tiện lợi

Sự Quyền năng

VS

Keras `model.fit()'

Rất tiện lợi, nhưng đóng hộp. Không phải kiến trúc nào cũng xài được (VD: mạng GAN huấn luyện Generator 1 bước, Discriminator 1 bước).

Custom Training Loop

Mất công tự viết vòng lặp, nhưng phá vỡ mọi giới hạn. Cho phép tùy biến và can thiệp ở cấp độ sâu nhất.

Gemini Notebook

Code Lab
`tf.GradientTape(): Tính đạo hàm tự động

TensorFlow cung cấp GradientTape đóng vai trò 'băng ghi âm' lưu vết Tensor để lan truyền ngược (Backpropagation) tính đạo hàm tự động.

```python
def f(w1, w2):
    return 3 * w1 ** 2 + 2 * w1 * w2

w1, w2 = tf.Variable(5.), tf.Variable(3.)

with tf.GradientTape() as tape:
    z = f(w1, w2)

# Yeu cau Tape tinh dao ham Z theo bien w1, w2
gradients = tape.gradient(z, [w1, w2])
print(gradients)
# Ket qua: dz/dw1=36, dz/dw2=10
```

Đạo hàm chính xác: dz/dw1 và dz/dw2

Gemini Notebook

Code Lab

GradientTape nâng cao

- **Mặc định:** Chỉ theo dõi biến `tf.Variable` (Không theo dõi `tf.constant`).
- Khi gọi phương thức `gradient()`, bộ nhớ Tape tự động bị xóa!
- **Giải pháp:** Tính nhiều lần đạo hàm (VD: Hessian), dùng cờ `persistent=True`.

```
1  with tf.GradientTape(persistent=True) as tape:
2      z = f(w1, w2)
3
4  dz_dw1 = tape.gradient(z, w1)
5  dz_dw2 = tape.gradient(z, w2) # Văn OK
6
7  # Giai phong bò nhớ bằng tay (bat buóc)
8  del tape
```

Cẩm trịnh, nhớ chi lùng persistent=True

Hành động bắt buộc: Giải phóng bộ nhớ thủ công khi dùng **persistent=True**

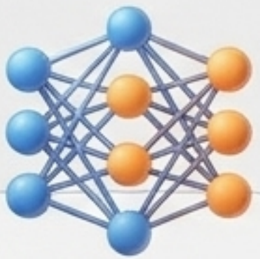
Gemini Notebook

Code Lab

Viết Vòng lặp huấn luyện - Khởi tạo

Bước 1: Chuẩn bị danh sách nguyên liệu. Tạo mô hình, cấu hình siêu tham số, Optimizer, Metric và Dữ liệu.

1. Khởi tạo Mô hình



```
l2_reg = keras.regularizers.l2(0.05)
model = keras.models.Sequential([
   keras.layers.Dense(30, activation="elu", kernel_regularizer=l2_reg),
   keras.layers.Dense(1, kernel_regularizer=l2_reg)
])
```

2. Chuẩn bị Dữ liệu

```
batch_size = 32
train_set = tf.data.Dataset.from_tensor_slices((X_train, y_train))
train_set = train_set.shuffle(len(X_train)).batch(batch_size)
```

3. Khởi tạo Optimizer

```
optimizer = keras.optimizers.Nadam(learning_rate=0.01)
```

4. Khởi tạo Loss & Metrics

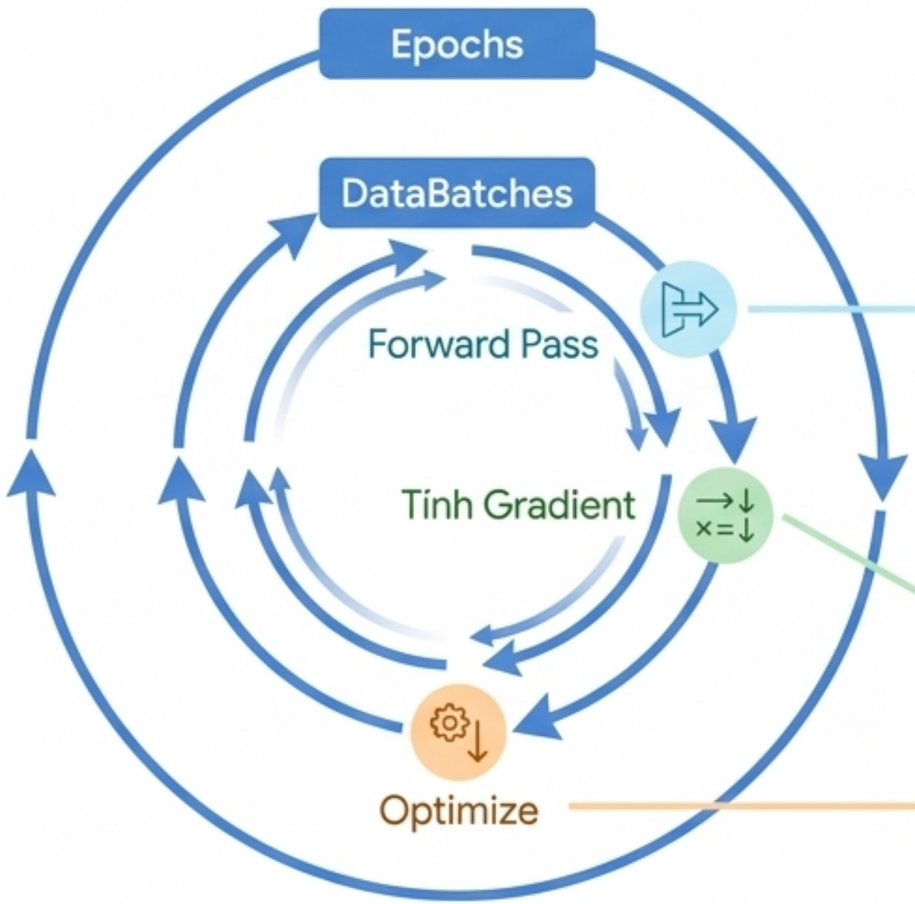
```
loss_fn = keras.losses.mean_squared_error
mean_loss = keras.metrics.Mean()
```

Gemini Notebook

Code Lab

Viết Vòng lặp huấn luyện - Thực thi (Epochs & Steps)

Lặp qua Epochs, trong Epochs lặp qua DataBatches.




```python
epochs = 5
for epoch in range(epochs):
  for step, (X_batch, y_batch) in enumerate(train_set):

    # 1. Forward pass ghi bang Tape
    with tf.GradientTape() as tape:
        y_pred = model(X_batch, training=True)
        main_loss = tf.reduce_mean(loss_fn(y_batch, y_pred))
        loss = tf.add_n([main_loss] + model.losses)

    # 2. Tinh Gradient theo model variables
    gradients = tape.gradient(loss, model.trainable_variables)

    # 3. Kich hoat phuong thuc cap nhat trong so (Optimize)
    optimizer.apply_gradients(zip(gradients,
model.trainable_variables))
```

Gemini Notebook

Code Lab

Cấu trúc hoàn chỉnh và Những lưu ý quan trọng

Khi tự viết vòng lặp, BẠN phải chịu trách nhiệm đảm bảo mọi thứ chính xác. Đừng quên 3 điểm chết người này:

Truyền cờ `training=True`

Bắt buộc truyền vào lệnh dự đoán của model để các lớp như BatchNormalization và Dropout hoạt động chính xác trong quá trình huấn luyện.

Quản lý Metrics thủ công

Phải liên tục theo dõi bằng `update_state(loss)`, in ra màn hình vào cuối mỗi Epoch và quan trọng nhất: gọi `reset_states()` để tránh cộng dồn sai.

Xử lý biến hệ thống (Non-trainable variables)

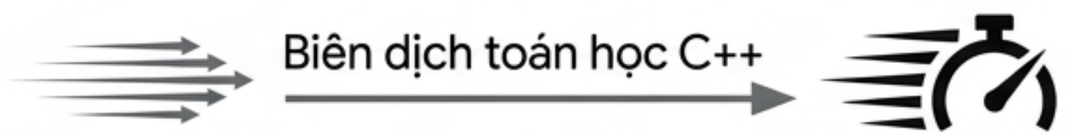
Tự tay viết logic cập nhật các biến không thể huấn luyện, ví dụ như giá trị trung bình trượt (moving averages) của lớp Batch Norm.

Gemini Notebook

Tối ưu hóa Vòng lặp với **@tf.function**

Vòng lặp `for` Python qua các DataBatches cực kỳ chậm. **Lời khuyên: Đóng gói toàn bộ quá trình xử lý 1 Bước (1 Step)** thành **@tf.function** để tăng tốc biên dịch toán học C++.

```python
@tf.function
def train_step(X_batch, y_batch):
    with tf.GradientTape() as tape:
        y_pred = model(X_batch, training=True)
        main_loss = tf.reduce_mean(loss_fn(y_batch, y_pred))
        loss = tf.add_n([main_loss] + model.losses)
    gradients = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(gradients, model.trainable_variables))

# Trong vong lap chi goi train_step(X_batch, y_batch)
```



Gemini Notebook

Tóm tắt: **4 Trụ cột TensorFlow Nâng cao**

Cốt lõi TensorFlow

Đại lượng Tensors cực kỳ giống NumPy, tối ưu hóa sức mạnh tính toán song song trên đa phần cứng (GPU/TPU).

Tùy biến Keras

Cho phép người học và nghiên cứu làm mới hoàn toàn và tự xây dựng các Losses, Metrics, Layers, Layers, Models chuyên biệt.

AutoGraph `@tf.function`

Giải pháp dung hòa hoàn hảo: Giải quyết sự linh hoạt của kịch bản Python với tốc độ biên dịch cực nhanh của đồ thị C++.

`tf.GradientTape`

Đưa quyền kiểm soát hoàn toàn tới luồng huấn luyện, mở khóa khả năng tính đạo hàm tự động cho các nghiên cứu mạng nơ-ron phức tạp nhất.

Gemini Notebook